

Trabajo Práctico 2

EDyA II

Secuencias

Integrantes:

Yuvvone, Cato

Altobello, Juliana

Becerra, Nicolás

28 de junio de 2025

Consigna

b) Para la implementación dada en (a), dar la especificación de costos (trabajo y profundidad) de las funciones `mapL`, `reduceS` y `scanS`. No es necesario demostrarlo.

Especificación de costos de `mapL`

Implementación

```
mapL :: (a -> b) -> [a] -> [b]
mapL _ [] = []
mapL f (x:xs) = let (y, ys) = f x ||| mapL f xs
                  in y:ys
```

Trabajo de `mapL` (W)

Para una lista, el trabajo se define como:

$$W(\text{mapL } f []) = k_1$$

$$W(\text{mapL } f (x : xs)) = W(f x) + W(\text{mapL } f xs) + k_2$$

Donde k_1 y k_2 son constantes que representan operaciones básicas. Luego:

$$W(\text{mapL } f (x : xs)) = \sum_{i=0}^{|xs|-1} W(f xs_i) + |xs| \cdot k_2$$

Por lo tanto:

$$W(\text{mapL } f (x : xs)) = O\left(\sum_{i=0}^{|xs|-1} W(f xs_i)\right)$$

Profundidad de `mapL` (S)

La profundidad la calculamos como:

$$S(\text{mapL } f []) = k_1$$

$$S(\text{mapL } f (x : xs)) = \max(S(f x), S(\text{mapL } f xs)) + k_2$$

Luego,

$$S(\text{mapL } f (x : xs)) = \max_{i=0}^{|xs|-1} S(f xs_i) + |xs| \cdot k_2$$

Por lo tanto:

$$S(\text{mapL } f (x : xs)) = O\left(\max_{i=0}^{|xs|-1} S(f xs_i) + |xs|\right)$$

Especificación de costos de `contrL`

Implementación

```
contrL :: (a -> a -> a) -> [a] -> [a]
contrL f [] = []
contrL f [x] = [x]
contrL f (x1:x2:xs) = let (y,ys) = (f x1 x2) ||| (contrL f xs)
                       in y:ys
```

Trabajo de `contrL` (W)

Tenemos que:

$$W(\text{contrL } f \ []) = k_1$$

$$W(\text{contrL } f [x]) = k_2$$

$$W(\text{contrL } f (x_1 : x_2 : xs)) = W(f x_1 x_2) + W(\text{contrL } f xs) + k_3$$

Donde k_1 , k_2 y k_3 son constantes. Luego:

$$W(\text{contrL } f (x_1 : x_2 : xs)) = \sum_{i=0}^{\lfloor \frac{|xs|}{2} \rfloor} W(f xs_{2i} xs_{2i+1}) + \left\lfloor \frac{|xs|}{2} \right\rfloor \cdot k_3$$

Por lo tanto:

$$W(\text{contrL } f (x_1 : x_2 : xs)) = O\left(\sum_{i=0}^{\lfloor \frac{|xs|}{2} \rfloor} W(f xs_{2i} xs_{2i+1})\right)$$

Profundidad de `contrL` (S)

La profundidad la calculamos como:

$$S(\text{contrL } f \ []) = k_1$$

$$S(\text{contrL } f [x]) = k_2$$

$$S(\text{contrL } f (x_1 : x_2 : xs)) = \max(S(f x_1 x_2), S(\text{contrL } f xs)) + k_3$$

Luego,

$$S(\text{contrL } f (x_1 : x_2 : xs)) = \max_{i=0}^{\lfloor \frac{|xs|}{2} \rfloor} S(f xs_{2i} xs_{2i+1}) + \left\lfloor \frac{|xs|}{2} \right\rfloor \cdot k_3$$

Por lo tanto:

$$S(\text{contrL } f (x_1 : x_2 : xs)) = O\left(\max_{i=0}^{\lfloor \frac{|xs|}{2} \rfloor} S(f xs_{2i} xs_{2i+1}) + |xs|\right)$$

Especificación de costos de `reduceL`

Implementación

```
reduceL :: (a -> a -> a) -> a -> [a] -> a
reduceL f e [] = e
reduceL f e [x] = f e x
reduceL f e xs = reduceL f e (contrL f xs)
```

Trabajo de `reduceL` (W)

$$W(\text{reduceL } f e \ []) = k_1$$

$$W(\text{reduceL } f e [x]) = W(f e x) + k_2$$

$$W(\text{reduceL } f e xs) = W(\text{reduceL } f e (\text{contrL } f xs)) + W(\text{contrL } f xs) + k_3$$

Por lo tanto:

$$W(\text{reduceL } f e xs) = O\left(\sum_{(f \ x \ y) \in \text{Or}(f, e, xs)} W(f \ x \ y)\right)$$

Profundidad de reduceL (S)

$$\begin{aligned}
S(\text{reduceL } f \ e \ []) &= k_1 \\
S(\text{reduceL } f \ e \ [x]) &= S(f \ e \ x) + k_2 \\
S(\text{reduceL } f \ e \ xs) &= S(\text{contrL } f \ xs) + S(\text{reduceL } f \ e \ (\text{contrL } f \ xs)) + k_3
\end{aligned}$$

Por lo tanto:

$$S(\text{reduceL } f \ e \ xs) = O\left(\max_{(f \ x \ y) \in \text{Or}(f, e, xs)} S(f \ x \ y) \cdot \log(|xs|) + |xs|\right)$$

Especificación de costos de scanL

Implementación

```

scanL :: (a -> a -> a) -> a -> [a] -> ([a], a)
scanL f b [] = ([], b)
scanL f b [x] = ([b], f b x)
scanL f b s = let (s', r) = scanL f b (contrL f s)
                in (expandL f b s s', r)

```

Trabajo de scanL (W)

Definimos el trabajo como:

$$\begin{aligned}
W(\text{scanL } f \ b \ []) &= k_1 \\
W(\text{scanL } f \ b \ [x]) &= W(f \ b \ x) + k_2 \\
W(\text{scanL } f \ b \ s) &= W(\text{contrL } f \ s) + W(\text{scanL } f \ b \ (\text{contrL } f \ s)) + W(\text{expandL } f \ b \ s \ s') + k_3
\end{aligned}$$

Donde:

- $W(\text{contrL } f \ s) = O\left(\sum_{i=0}^{\lfloor n/2 \rfloor - 1} W(f \ s_{2i} \ s_{2i+1})\right)$
- $W(\text{expandL } f \ b \ s \ s') = O(n)$ (asumiendo que procesa cada elemento una vez)

La recurrencia para el trabajo es:

$$W(n) = W\left(\frac{n}{2}\right) + O(n)$$

Cuyo resultado es:

$$W(\text{scanL } f \ b \ s) = O\left(\sum_{(f \ x \ y) \in \text{Or}(f, b, s)} W(f \ x \ y) + n\right)$$

Profundidad de scanL (S)

Definimos la profundidad como:

$$\begin{aligned}
S(\text{scanL } f \ b \ []) &= k_1 \\
S(\text{scanL } f \ b \ [x]) &= S(f \ b \ x) + k_2 \\
S(\text{scanL } f \ b \ s) &= S(\text{contrL } f \ s) + S(\text{scanL } f \ b \ (\text{contrL } f \ s)) + S(\text{expandL } f \ b \ s \ s') + k_3
\end{aligned}$$

Donde:

- $S(\text{contrL } f \ s) = O\left(\max_{i=0}^{\lfloor n/2 \rfloor - 1} S(f \ s_{2i} \ s_{2i+1}) + \log n\right)$
- $S(\text{expandL } f \ b \ s \ s') = O(\log n)$ (asumiendo paralelismo en la expansión)

La recurrencia para la profundidad es:

$$S(n) = S\left(\frac{n}{2}\right) + O(\log n)$$

Cuyo resultado es:

$$S(\text{scanL } f \ b \ s) = O\left(\max_{(f \ x \ y) \in \text{Or}(f, b, s)} S(f \ x \ y) \cdot \log n + \log^2 n\right)$$

Consigna:

d) Muestre que la implementación dada en (c) verifica la especificación de costos para la profundidad de la operación **reduceS**.

Veamos la profundidad de **reduceS**:

La profundidad de **nth** es constante.

La profundidad de **tabulate** es

$$O\left(\max_{i=0}^{n-1} S(f \ i)\right).$$

En nuestra implementación de **contra**:

```
contraA :: (a -> a -> a) -> Arr a -> Arr a
contraA f arr = tabulateA faux m
  where
    faux i = if i == mid
              then nthA arr (n-1)
              else f (nthA arr (i*2)) (nthA arr (i*2+1))
    mid = n div 2
    n   = lengthA arr
    m   = if n mod 2 == 0 then mid else mid + 1
```

Se usa **tabulateA** con argumento **faux** y tamaño $m = \lceil n/2 \rceil$; la profundidad de **faux** es $O(S(f))$. Por lo tanto,

$$S(\text{contraA } f \ arr) = O\left(\max_{i=0}^{\lceil n/2 \rceil - 1} S(f \ arr_{2i} \ arr_{2i+1})\right).$$

La función **reduceA** se definió así:

```
reduceA :: (a -> a -> a) -> a -> Arr a -> a
reduceA f e arr =
  case lengthA arr of
    0 -> e
    1 -> f e (nthA arr 0)
    _ -> reduceA f e (contraA f arr)
```

Cada llamada a **contraA** reduce a la mitad la longitud del arreglo, de modo que el número de iteraciones es $O(\log |arr|)$.

Y en cada iteración llamo a **contraA**, por lo tanto la profundidad queda

$$S(\text{reduceA } f \ e \ arr) = O\left(\log |arr| \cdot \max_{(f(x,y) \in \mathcal{O}_s(f,e,arr))} S(f(x,y))\right)$$